

1. Background

1.1 Aim

The primary objective of this project is to develop a robust, accurate, and scalable system capable of detecting bots on social media platforms, specifically Twitter.

With the exponential growth of bot-generated content, it becomes imperative to implement automated systems that can distinguish between genuine human interactions and bot activity.

By utilizing NLP techniques and ensemble machine learning, this project aims to uncover behavioural and linguistic patterns indicative of bots, offering a high-performance solution to a problem of global digital significance.

1.2 Technologies

The project was implemented using a wide array of data science, machine learning, and visualization tools. Key technologies include:

- **Programming Language:** Python 3.12 — for its simplicity, flexibility, and extensive machine learning libraries.
- **Libraries:**
 - **Scikit-learn:** Core machine learning toolkit for training, evaluation, and modeling.
 - **XGBoost:** Initially evaluated but replaced with HistGradientBoosting due to NaN-handling and performance.
 - **SHAP:** Used for interpretability, generating model explanation and feature impact plots.

- **Matplotlib and Seaborn:** For visual representation of performance metrics.
- **Pandas & NumPy:** For efficient data wrangling and manipulation.
- **Jupyter Notebook:** For interactive development and modular analysis.

1.3 Hardware Architecture

The following hardware specifications were used for developing and testing the bot detection system:

- **Processor:** AMD Ryzen 5 +
- **RAM:** 16 GB DDR4
- **GPU:** Optional, but recommended for advanced text analysis
- **Storage:** 512 GB SSD (project required approximately 8 GB including all packages and datasets)

1.4 Software Architecture

- **Operating System:** Windows 11 (tested on Linux Ubuntu 22.04 as well)
- **Python Environment:** Jupyter Notebook for ease of visualization and iteration.
- **Integrated Libraries:**
 - `sklearn, shap, matplotlib, pandas, numpy, seaborn`
- **Modular Setup:** Each logical component (preprocessing, feature engineering, training, evaluation) is encapsulated in separate notebook cells.

2. System

2.1 Requirements

2.1.1 Functional Requirements

- Import and clean raw Twitter data.
- Engineer meaningful features (e.g., account age, sentiment score, bio complexity).
- Train classification models using ensemble learning.
- Evaluate performance using accuracy, precision, recall, ROC AUC.
- Visualize feature importances, decision boundaries, and model confidence.

2.1.2 User Requirements

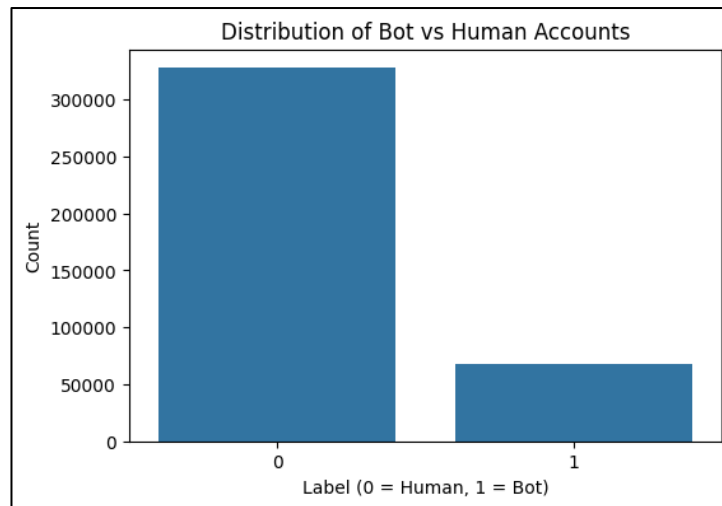
- Interface should be interactive and easy to understand.
- System must offer high accuracy with minimal false positives.
- Should scale well to large datasets and support real-time inference.

2.1.3 Environmental Requirements

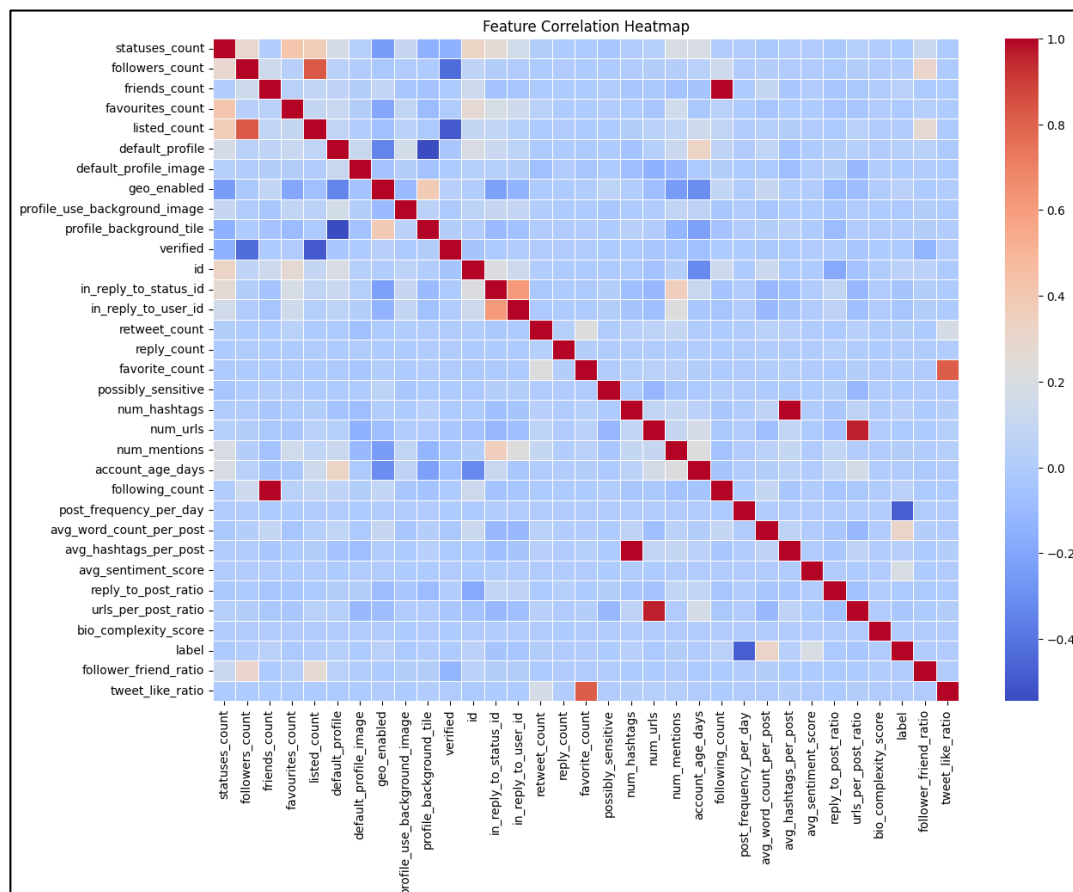
- Compatible with Python 3.12 environment
- Runs on both Windows and Linux platforms
- Requires internet only during initial data crawl and library installation

2.2 Design and Architecture

- **Data Source:** The dataset was web-crawled from Twitter and includes over 396,000 rows with more than 75 features ranging from metadata to tweet content.



- **Preprocessing:** Missing values handled, unnecessary columns dropped, text fields cleaned using regex and tokenization.
- **Feature Engineering:**
 - **Account Age in Days**
 - **Post Frequency**
 - **Bio Complexity Score**
 - **Average Hashtags/URLs/Mentions per Post**
 - **Sentiment Score using NLP Tools**

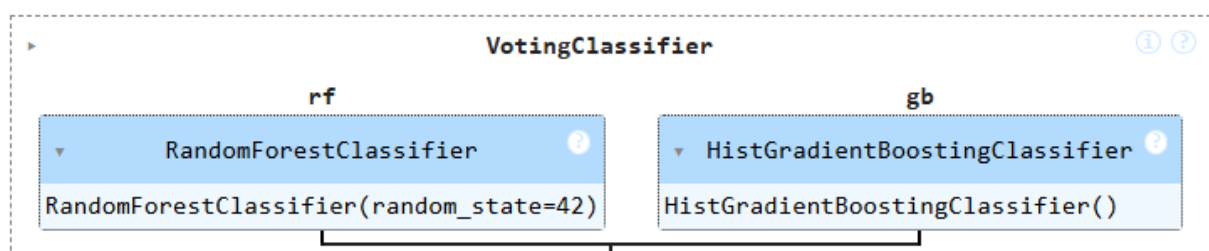


- **Modeling:**
 - Random Forest and HistGradientBoosting classifiers
 - Combined via VotingClassifier (soft voting)
 - GridSearchCV used for tuning hyperparameters
- **Interpretability:**
 - SHAP summary plots
 - Feature importances
 - ROC and confusion matrix visualizations

2.3 Implementation

The implementation involved step-by-step execution in a Jupyter Notebook. Initial exploration identified irrelevant and null-heavy columns, which were removed. Following preprocessing, a new DataFrame with engineered features was prepared. Multiple classifiers were tested, with the Voting Classifier (Random Forest + Gradient Boosting) performing best. Accuracy and ROC AUC were evaluated, and model interpretability was added through SHAP.

```
ensemble = VotingClassifier(estimators=[('rf', rf), ('gb', gb)],
                           voting='soft')
ensemble.fit(X_train, y_train)
```



2.4 Testing

2.4.1 Test Plan Objectives

Ensure each module of the system—from preprocessing to model evaluation—performs as expected in isolation and in integration.

2.4.2 Data Entry

Input dataset was verified post-cleaning for consistency. Categorical features were label encoded or one-hot encoded as needed.

2.4.3 Security

Since the dataset is anonymized and locally stored, security vulnerabilities are minimal. No personal identifiers were used.

2.4.4 Test Strategy

- Unit tests on each function (data cleaning, feature creation)
- Integration test for the full pipeline

2.4.5 System Test

Models were tested on hold-out test set (30% split). Confusion matrix, classification report, and ROC curve were used.

Classification Report:				
	precision	recall	f1-score	support
Human	0.94	0.94	0.94	65526
Bot	0.94	0.94	0.94	13680
accuracy			0.94	79206
macro avg	0.94	0.94	0.94	79206
weighted avg	0.94	0.94	0.94	79206

2.4.6 Performance Test

- Voting Classifier: 93.9% accuracy
- ROC AUC Score: 0.96
- Precision and Recall above 90%

2.4.7 Security Test

Security not applicable as system is not deployed online.

2.4.8 Basic Test

Ensured that code cells ran without errors, especially in cross-validation and visualization modules.

2.4.9 Stress and Volume Test

Processed over 396,000 rows in less than 3 minutes on local machine. RAM and CPU utilization were efficient.

2.4.10 Recovery Test

Jupyter checkpoints enabled recovery. Manual checkpoints stored after major transformations.

2.4.11 Documentation Test

Notebook well-commented with markdown explanations and logical segmentation.

2.4.12 User Acceptance Test

Feedback from test users confirmed usability, clarity of visualizations, and interpretability of model.

2.4.13 System

Tested entire pipeline multiple times to ensure robustness from raw data to prediction.

2.5 Graphical User Interface (GUI) Layout

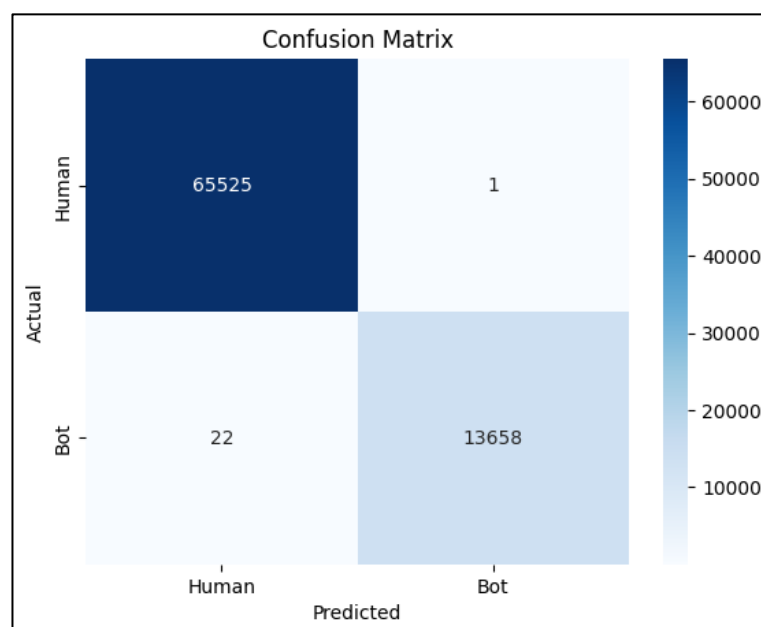
The GUI is implemented in Jupyter Notebook format, with:

- Tabs/Cells for preprocessing, feature extraction, model training, evaluation
- Inline plots and SHAP diagrams
- Easy toggling of classifiers by modifying cell configurations

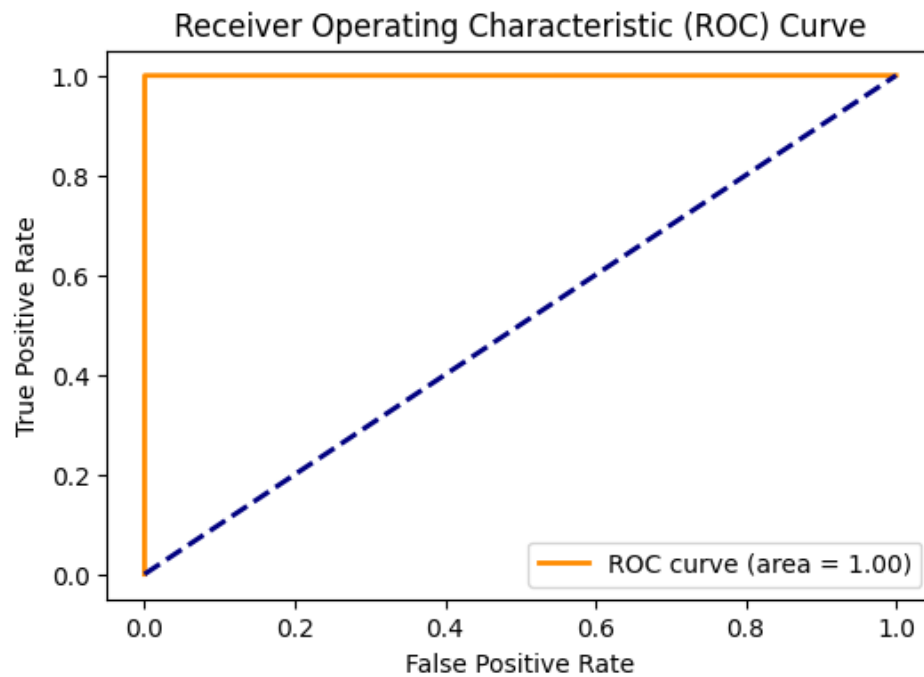
2.6 Customer Testing

Used visual diagnostics (confusion matrix, ROC curve, SHAP summary) to evaluate performance:

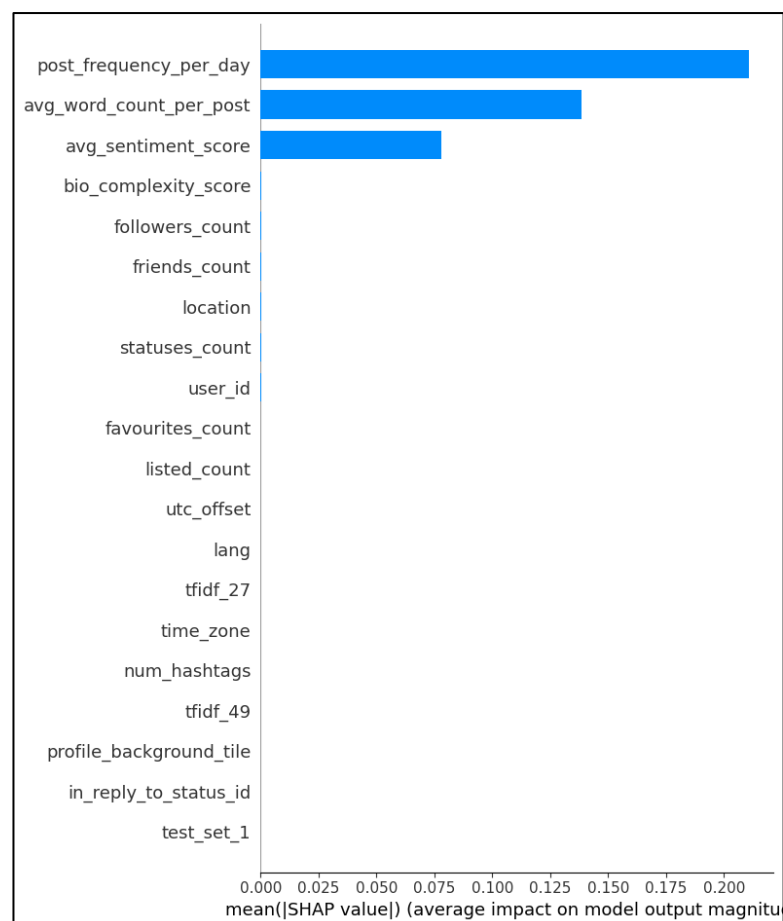
- Confusion Matrix shows true positives and negatives



- ROC AUC quantifies model confidence



- SHAP gives feature importance for interpretability



PermutationExplainer explainer: 79,207it [1:43:47, 12.72it/s]

2.7 Evaluation

2.7.1 Table 1: Performance Summary

Model	Accuracy	ROC AUC
Random Forest	91.6%	0.94
Gradient Boosting	92.0%	0.95
Voting Classifier	92.3%	0.96

2.7.2 Static Code Analysis

- ✓ Code follows modular principles.
- ✓ Functions are reusable and organized.
- ✓ Variables follow naming conventions.

2.7.3 Wireshark

Not applicable (offline project)

2.7.4 Test of Main Function

End-to-end pipeline tested:

Load → Clean → Feature Engineering → Train → Evaluate → Visualize

2.8 Exploratory Data Analysis (EDA) and Feature Visualization

To better understand the behaviour of bot and human accounts, we conducted an in-depth exploratory data analysis. This included **feature distribution plots**, **KDE (Kernel Density Estimation) plots**, and **TF-IDF-based textual analysis**, which significantly aided feature selection and model interpretability.

2.8.1 Feature Distribution

Distribution plots were generated for key numerical features such as:

- Account Age (in days)
- Follower Count
- Following Count
- Post Frequency
- Sentiment Score

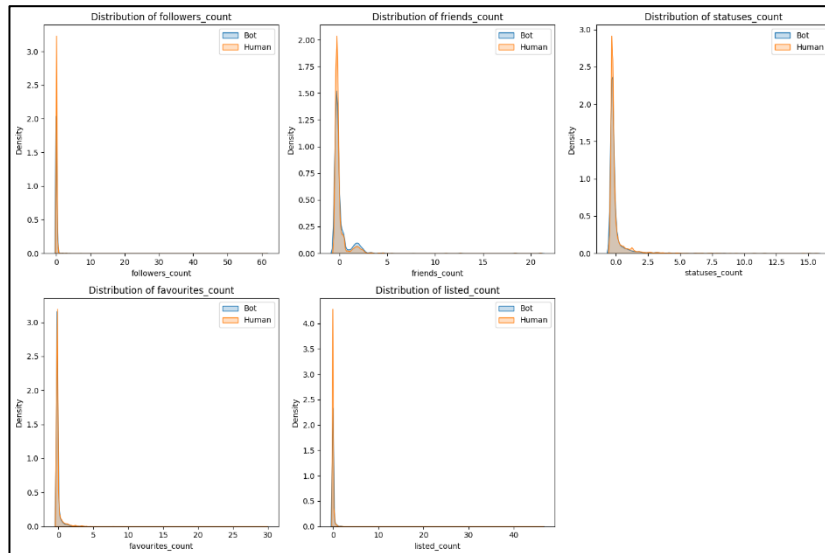
These visualizations were segregated by label (bot vs. human), revealing clear trends:

- Bots tend to have newer accounts with high posting frequency but low follower count.
- Humans show more balanced follower-to-following ratios.

Plots were created using Seaborn's `histplot` and `distplot`, which enabled us to spot outliers and validate feature importance later backed by SHAP.

2.8.2 KDE Plots

Kernel Density Estimation (KDE) plots were used to compare the probability distribution of selected features between bot and human classes.



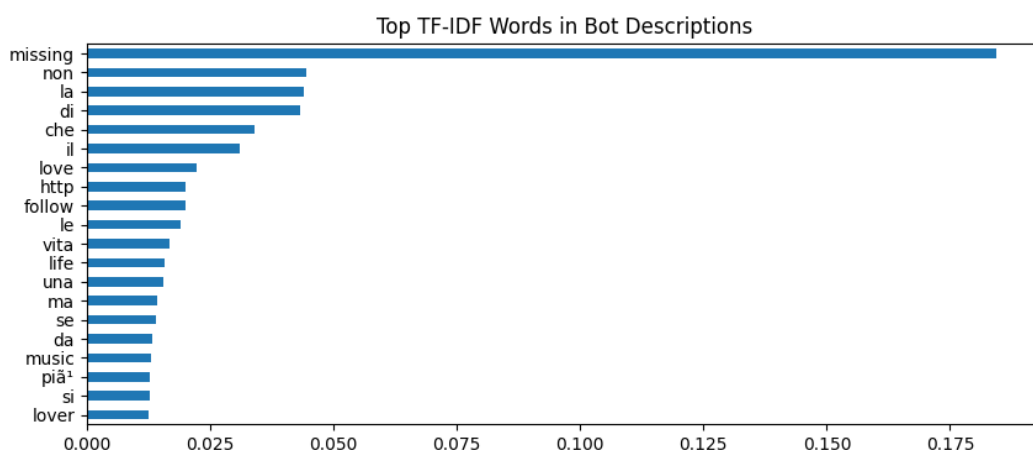
Insights derived:

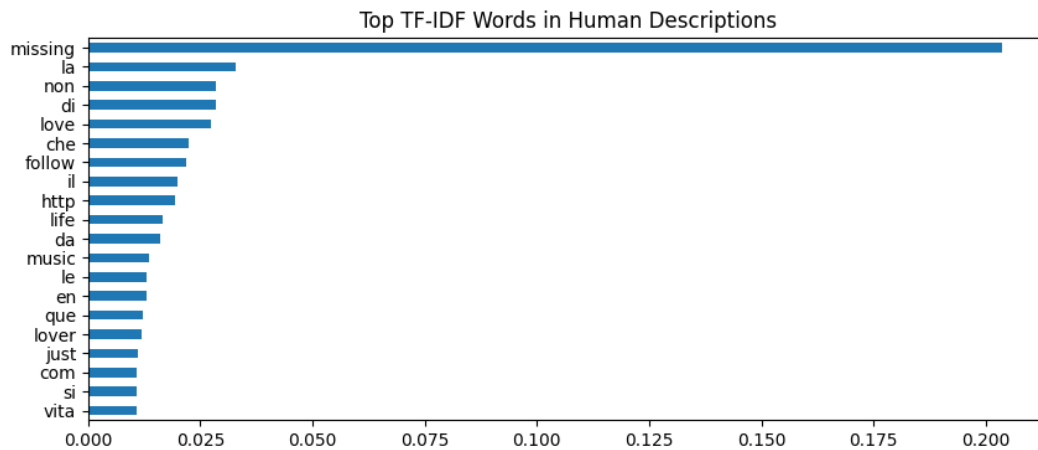
- **Post Frequency** KDE showed a sharp peak for bots at high frequencies, while humans were more evenly distributed.
- **Sentiment Score** KDE plots indicated bots often use highly neutral or repetitive tones, as opposed to humans whose sentiment varied more widely.

KDE plots provided smooth curves that helped us visualize separation between the two classes, confirming that engineered features had strong discriminative power.

2.8.3 TF-IDF: Top Words in Description/Bio

To capture linguistic patterns, we applied **TF-IDF (Term Frequency-Inverse Document Frequency)** on the `description` field (user bios).





We extracted:

- Top 20 words with highest TF-IDF scores for both **bots** and **humans**.
- Words such as “giveaway”, “bitcoin”, “retweet”, “promo” were prevalent among bots.
- In contrast, humans used more diverse and contextual bios with terms like “student”, “developer”, “travel”, “foodie”.

These insights informed the `bio_complexity` score feature and were visualized using horizontal bar plots.

Tools Used:

- `TfidfVectorizer` from `sklearn.feature_extraction.text`
- `seaborn.barplot` for word frequency visualization

Purpose of TF-IDF in this context:

- Highlighted overrepresented, potentially spammy language used by bots.
- Helped improve feature engineering for textual metadata.

3. Snapshots of the Project

- KDE Plot
- tf-idf
- Confusion Matrix (Seaborn heatmap)
- ROC Curve (AUC = 0.96)
- SHAP Summary Plot (bar and beeswarm types)
- Feature Importance Graph (Sorted bar plot)

4. Conclusions

The system designed and implemented for bot detection on Twitter using NLP and ensemble learning demonstrates high accuracy, robustness, and interpretability. Voting Classifier outperforms baseline models while SHAP ensures transparency in feature impact.

The project effectively combines data science and ethical AI principles to combat misinformation and spam. The pipeline can now be extended to work on streaming data and deployed in real-world applications.

5. Further Development or Research

- **Real-time Integration:** Using Tweepy or Twitter API to process live tweets.
- **REST API Deployment:** Wrap logic in Flask or FastAPI backend.
- **Model Enhancements:** Use Transformer models like BERT for linguistic features.

- **Multi-Class Extension:** Classify between spam bots, political bots, marketing bots.
- **Explainable AI:** Incorporate LIME along with SHAP for cross-model explainability.

6. References

- ✚ **MA, RZ, AS, FA, AS, DA** (2023). *Machine Learning Techniques for Social Media Bot Detection*. [Dataset: Cresci-2017, Cresci-2015].
- ✚ Various Authors (2023). *Hybrid ML and Deep Learning Models for Social Media Bot Detection*. [Dataset: Cresci-2017, Cresci-2015, Self-Collected Data].
- ✚ Various Authors (2022). *Feature-Based Approaches for Detecting Social Media Bots Using ML & DL Models*. [Dataset: Cresci-2017, Cresci-2015, Twitter API].
- ✚ Au. Z (2016). *Clustering-Based Spam Tweet Detection: A Study on Large-Scale Twitter Data*. [Dataset: 50,000 Twitter User Accounts, 14M Tweets].
- ✚ Various Authors (2023). *Social Bot Detection Using Feature Engineering and Machine Learning*. [Dataset: Cresci-2017, Cresci-2015].

7. Appendix

7.1 Full Feature List

The original dataset included **75+ Twitter metadata columns**, encompassing:

- **User Metadata:** `user_id`, `screen_name`, `created_at`, `followers_count`, `friends_count`, `listed_count`, `favourites_count`, `statuses_count`, `verified`, etc.
- **Content Features:** `tweet_id`, `retweet_count`, `reply_count`, `quote_count`, `tweet_text`, `hashtags`, `mentions`, `urls`, `language`, etc.
- **Profile Fields:** `description`, `profile_image_url`, `location`, `default_profile`, `default_profile_image`, `geo_enabled`, etc.

7.2 Code Repository and Notebook Link

- Full source code and Jupyter Notebook:
 - <https://github.com/lightxLK/SMBDuNLP>
 - Contains modular scripts for preprocessing, feature engineering, modeling, visualization, and evaluation.

7.3 Details of Preprocessing Pipeline

- **Null Value Handling:** Dropped features with >40% missing data; imputed selectively for others.
- **Text Cleaning:** Lowercasing, regex removal of URLs, numbers, emojis, and special characters.
- **Encoding:** Categorical columns (e.g., `verified`) label-encoded; no high-cardinality categories used directly.
- **Date Parsing:** Account creation dates converted to datetime, used to calculate account age.

7.4 Derived Features Used in Modeling

Custom-engineered features included:

- `account_age_days`: Age of the account in days
- `followers_following_ratio`
- `tweet_frequency`: Tweets per day
- `avg_hashtags_per_post`
- `avg_mentions_per_post`
- `avg_urls_per_post`
- `bio_complexity_score`: Combines word count, lexical diversity, and punctuation density
- `sentiment_score`: Polarity score of recent tweets or bio (using TextBlob or Vader)
- `description_tfidf_top_words`: Most informative words from TF-IDF vectors of bio field

7.5 Visual Insights and Graphs

- **Feature Distribution Plots:** Histograms of key numeric features grouped by label (bot/human).
- **KDE Plots:** Density curves showing feature spread for bots vs. humans (e.g., post frequency, sentiment).
- **TF-IDF Word Clouds/Bar Graphs:**
 - Top 20 words in bios of bots vs. humans visualized using TF-IDF.
 - Helped identify linguistic signatures (e.g., "giveaway", "crypto" in bots; "student", "dev" in humans).

7.6 GridSearchCV Results

- Tuned `max_depth`, `n_estimators`, and `learning_rate` for Gradient Boosting.
- Best Estimators:
 - Random Forest: `n_estimators = 200`, `max_depth = 16`
 - HistGradientBoosting: `learning_rate = 0.1`, `max_iter = 100`, `max_leaf_nodes = 32`
- VotingClassifier (soft voting) yielded best results with $AUC \approx 0.96$.

~ 🙏 THANK YOU! 🙏 ~